AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 08.12.2026

Lab 09

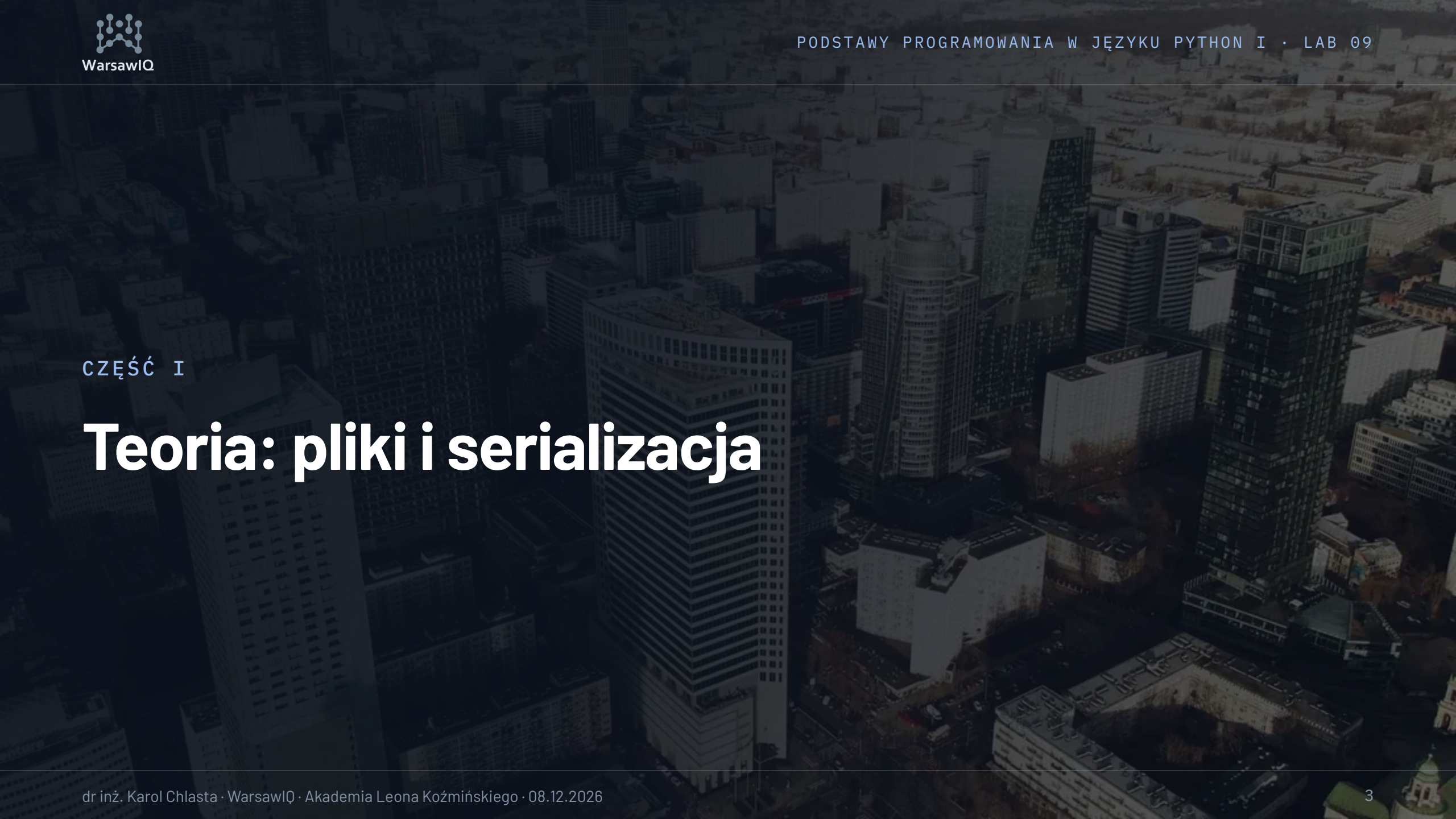
Praca z plikami

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć, **czym jest plik**: bajty, kodowanie znaków, tryby otwarcia
- Czytać i zapisywać pliki tekstowe, bezpiecznie zamykać je przez `with`
- Obsługiwać brak pliku i błędne dane (`try` / `except`)
- Zapisywać dane jako **JSON**; poznać `pickle` i jego ograniczenia
- Efekty z sylabusa: **U1** (obsługa plików), **U2**, **KS1**



CZĘŚĆ I

Teoria: pliki i serializacja

Plik to ciąg bajtów

- Dysk przechowuje **bajty** (liczby 0–255); znaczenie nadaje im format
- **Plik tekstowy** = bajty + **kodowanie znaków**, np. UTF-8
- **Plik binarny** = bajty interpretowane wprost (obraz, PDF, `.pyc`)

```
"ł".encode("utf-8")      # b'\xc5\x82' – jedna litera, dwa bajty
b"\xc5\x82".decode()     # 'ł'
```

> **Zasada:** przy plikach tekstowych zawsze podawaj `encoding="utf-8"`. W Pythonie do wersji 3.14 domyślne kodowanie zależy od systemu (na Windows bywa to `cp1250`); dopiero od 3.15 domyślne jest UTF-8 (PEP 686).

Tryby otwarcia

TRYB	ZNACZENIE	GDY PLIK NIE ISTNIEJE	GDY ISTNIEJE
`"r"`	odczyt (domyślny)	`FileNotFoundError`	czyta
`"w"`	zapis	tworzy	**czyści zawartość!**
`"a"`	dopisywanie	tworzy	dopisuje na końcu
`"x"`	zapis wyłącznie nowego	tworzy	`FileExistsError`
`"b"`	dodatek: tryb binarny	`"rb"`, `"wb"`	

```
with open("notatka.txt", "a", encoding="utf-8") as f:
    f.write("nowa linia\n")      # write NIE dodaje "\n" sam
```

`with`: plik zawsze zamknięty

```
with open("dane.txt", encoding="utf-8") as f:
    for line in f:                  # czyta linia po linii – oszczędza pamięć
        print(line.rstrip("\n"))
# tu plik jest już zamknięty – nawet jeśli w bloku wystąpił błąd
```

METODA	ZWRACA
`f.read()`	całą zawartość jako `str`
`f.readline()`	jedną linię (z `"\n"`)
`for line in f`	kolejne linie – **najlepszy wybór** dla dużych plików
`f.write(s)`	zapisuje tekst, zwraca liczbę znaków

- `with` to **menedżer kontekstu**: gwarantuje sprzątanie (zamknięcie pliku, zwolnienie zasobu)

Ścieżki i `pathlib`

```python
from pathlib import Path

p = Path("dane") / "oceny.csv"    # operator / łączy ścieżki – działa na każdym systemie
p.exists(), p.is_file()           # (True, True)
p.suffix, p.stem, p.name          # ('.csv', 'oceny', 'oceny.csv')
p.read_text(encoding="utf-8")     # cały plik jednym wywołaniem
Path.cwd()                        # bieżący katalog roboczy
```

- **Ścieżka względna** liczy się od katalogu, **z którego uruchomiono program** – nie od pliku `.py`
- Stąd częsty błąd: `FileNotFoundError`, choć plik „przecież jest”

Błędy przy pracy z plikami

```
try:
    with open(path, encoding="utf-8") as f:
        text = f.read()
except FileNotFoundError:
    text = ""
```

WYJĄTEK	KIEDY
`FileNotFoundError`	brak pliku przy odczycie
`PermissionError`	brak uprawnień
`UnicodeDecodeError`	złe kodowanie
`ValueError`	np. `float("abc")` przy przetwarzaniu treści

Serializacja: JSON

> **Serializacja** – zamiana obiektu w pamięci na ciąg znaków lub bajtów, który da się zapisać lub wysłać.

```python
import json
contacts = {"Ola": "600100200", "Łukasz": "500600700"}

with open("kontakty.json", "w", encoding="utf-8") as f:
    json.dump(contacts, f, ensure_ascii=False, indent=2)

with open("kontakty.json", encoding="utf-8") as f:
    contacts = json.load(f)
```

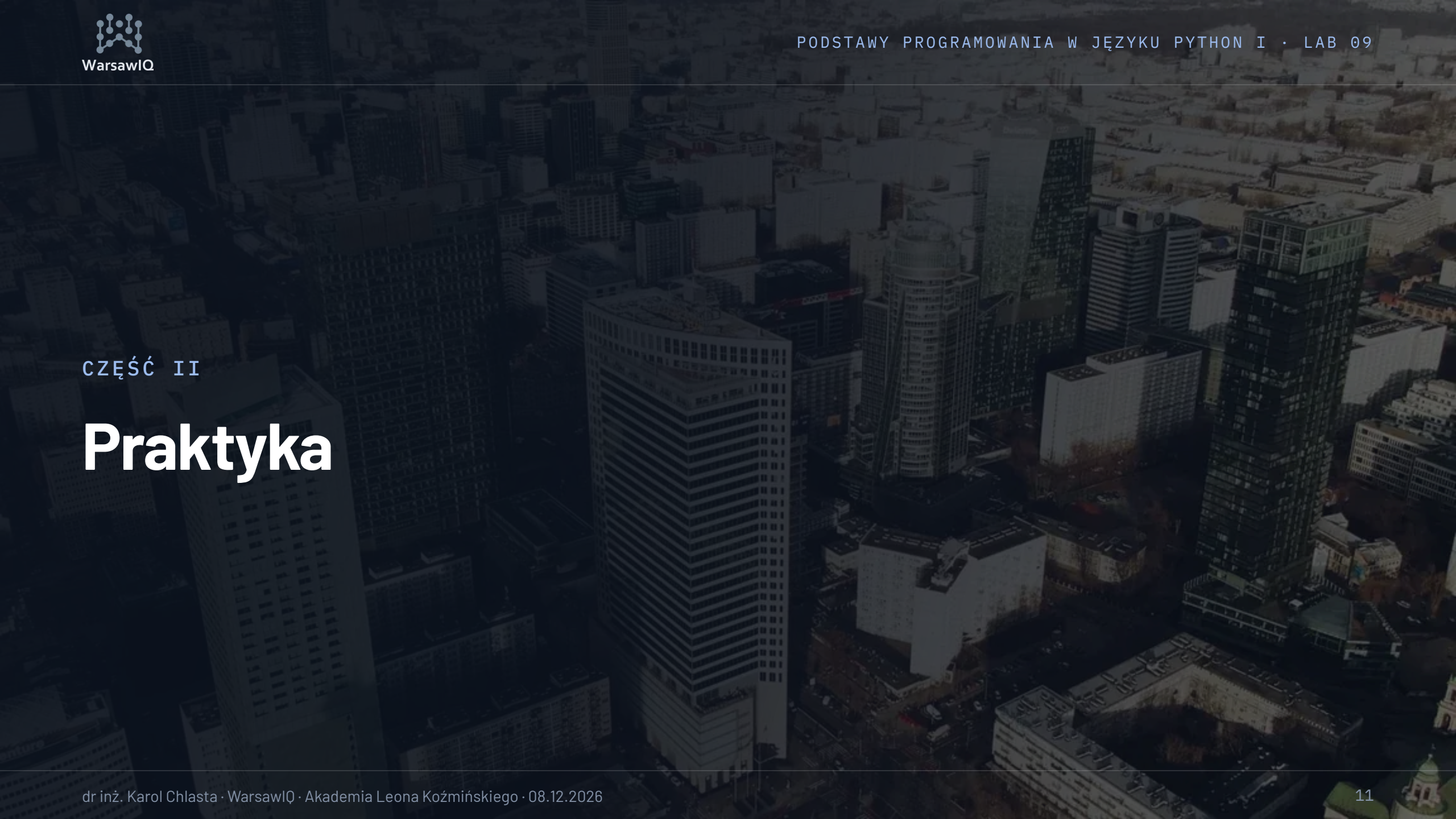
- JSON: tekstowy, czytelny, **niezależny od języka** – standard wymiany danych w API
- Typy: obiekt (`dict`), tablica (`list`), tekst, liczba, `true` / `false`, `null`

`pickle`: wygodny, ale niebezpieczny

```
import pickle
with open("dane.pkl", "wb") as f:      # tryb binarny!
    pickle.dump({"oceny": [5, 4.5]}, f)
```

	JSON	PICKLE
Format	tekst	binarny
Języki	dowolne	tylko Python
Typy	podstawowe	prawie każdy obiekt Pythona
Bezpieczeństwo	bezpieczny	**wczytanie może uruchomić dowolny kod**

> Dokumentacja Pythona: *„Never unpickle data that could have come from an untrusted source”*.
> Domyślnie wybieraj **JSON**.



CZĘŚĆ II

Praktyka

Wzorzec: wczytaj → zmień → zapisz

```
def add_contact(path, name, phone):
    contacts = load_contacts(path)   # {} gdy pliku brak
    contacts[name] = phone
    save_contacts(contacts, path)
```

- Tak działa większość prostych „baz danych” w plikach (ustawienia, notatki, wyniki gier)
- Ryzyko: awaria w trakcie zapisu = uszkodzony plik. Poważne aplikacje zapisują do pliku tymczasowego i podmieniają go albo używają bazy danych

Ciekawostki 💾

- **UTF-8** zaprojektowali Ken Thompson i Rob Pike we wrześniu 1992 r. – według relacji Pike'a szkic powstał na papierowej podkładce w restauracji w New Jersey
- **JSON** spopularyzował Douglas Crockford na początku lat 2000.; dziś to standardy ECMA-404 i RFC 8259
- Każdy plik PNG zaczyna się od tych samych 8 bajtów: `89 50 4E 47 0D 0A 1A 0A` (`\x89PNG\r\n\x1a\n`) – sprawdź: `open("obraz.png", "rb").read(8)`
- Filozofia Uniksa: **„wszystko jest plikiem”** – także klawiatura i ekran (`stdin`, `stdout`)

Dzisiejsze zadania

	PLIK	ZADANIE
★	`zad1_logger.py`	logowanie zdarzeń z datownikiem
★	`zad2_suma_z_pliku.py`	suma liczb z pliku
★	`zad3_edytor.py`	prosty edytor tekstu
★	`zad4_json.py`	kontakty w JSON
☆	`zad5_menedzer.py`	menedżer plików

Sprawdzenie i oddanie

```
cd lab09_pliki
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 09"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- Po zajęciach – **0 pkt**, bez wyjątków

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 10
- M. Dawson, *Python dla każdego*, Helion 2015 – rozdz. 7
- Dokumentacja: *Reading and Writing Files* (*docs.python.org/3/tutorial/inputoutput.html*), moduły `json`, `pickle`, `pathlib`

Następny lab (15.12): moduły i pakiety